

Experiment Name

Write a program to identify unique identifiers from a C program using a basic lexical analyzer.

Objective

The objective of this experiment is to design and implement a basic lexical analyzer that scans a C program and detects valid identifiers. The program identifies unique identifiers while ignoring keywords, invalid tokens, and duplicates. This helps in understanding the working of lexical analysis in compiler design.

Algorithm

1. Start the program.
2. Read the input C program from a file or user input.
3. Define a list of C keywords (such as int, float, char, if, while, return, etc.).
4. Initialize an empty set or list to store unique identifiers.
5. Read the program line by line.
6. For each line, break the line into tokens using delimiters such as spaces, commas, semicolons, parentheses, and operators.
7. For each token:
 - (a) Check if the token starts with a letter (a–z or A–Z) or an underscore (_).
 - (b) Check if the remaining characters of the token contain only letters, digits, or underscores.
 - (c) If the token does not satisfy identifier rules, skip it.
 - (d) Compare the token with the list of C keywords.
 - (e) If the token matches a keyword, ignore it.
 - (f) Otherwise, check whether the identifier is already stored.
 - (g) If it is not present, add it to the list of unique identifiers.
8. Continue scanning until the entire program is processed.
9. Display all unique identifiers found in the C program.
10. Stop the program.

Source Code

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  bool isKeyword(const string &word)
4  {
5      static const vector<string> keywords = {
6
7          "auto", "break", "case", "char", "const", "continue", "default", "do", "double",
8
9          "else", "enum", "extern", "float", "for", "goto", "if", "int", "long", "register",
10         "return", "short", "signed", "sizeof", "static", "struct", "switch", "typedef",
11         "union", "unsigned", "void", "volatile", "while"
12     };
13
14     for (const auto &k : keywords)
15         if (word == k)
16             return true;
17
18     return false;
19 }
20
21 bool isIgnored(const string &word)
22 {
23     static const vector<string> ignored = {
24         "include", "stdio", "iostream", "printf", "scanf",
25         "cin", "cout", "endl"
26     };
27
28     for (const auto &w : ignored)
29         if (word == w)
30             return true;
31
32     return false;
33 }
34
35 bool isValidIdentifier(const string &word)
36 {
37     if (word.empty() || !(isalpha(word[0]) || word[0] == '_'))
38         return false;
39
40     for (char c : word)
41         if (!(isalnum(c) || c == '_'))
42             return false;
43
44     return true;
45 }
46
47 int main()
48 {
49     ifstream file("input.c");
50     map<string, int> frequency;
51     string word;
52
53     while (file >> word)
54     {
55         word = cleanToken(word);
56
57         if (isValidIdentifier(word) &&
58             !isKeyword(word) &&
59             !isIgnored(word))
60         {
61             frequency[word]++;
62         }
63     }
64
65     for (const auto &entry : frequency)
66     {
67         cout << left << setw(20)
68              << entry.first
69              << entry.second << "\n";
70     }
71
72     return 0;
73 }
```

Figure 1: Source code for lexical analyzer to detect unique identifiers

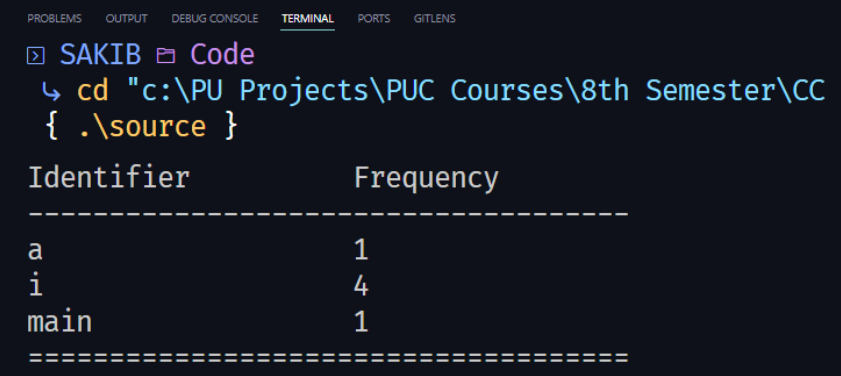
```

1  #include <stdio.h>
2
3  int main()
4  {
5      int i;
6      int a[10];
7      for(i = 1; i <= 10; i++)
8      {
9          printf("Bangladesh\n");
10     }
11
12     return 0;
13 }

```

Figure 2: Input code for lexical analyzer to detect unique identifiers

Output Screenshot



```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS
SAKIB Code
cd "c:\PU Projects\PUC Courses\8th Semester\CC"
{ .\source }

Identifier      Frequency
-----
a               1
i               4
main            1
=====

```

Figure 3: Output showing all unique identifiers extracted from the C program

Discussion

This experiment demonstrates the design of a simple lexical analyzer to extract identifiers from a C program. The analyzer scans the input, applies identifier rules, removes keywords, and ensures uniqueness using a suitable data structure. This experiment provides a clear understanding of tokenization, identifier recognition, and the importance of lexical analysis in the first phase of a compiler.